

Quiz Master - V1 Project Report

Name: Ambuj Pratap , **Roll Number:** 22f3002778 ,
Email: 22f3002778@ds.study.iitm.ac.in
Submission Date: February 28, 2025

Project Title: Quiz Master - V1

2. Project Details

- **Project Title:** Quiz Master - V1

Question Statement:

- "Develop a web-based quiz management system that allows users (admins and students) to create, manage, and attempt quizzes, track progress, and visualize performance using interactive charts."

- **Approach to the Problem Statement:**

I approached this project by first analyzing the requirements for a quiz management system, identifying key features (user authentication, quiz creation, attempt tracking, and performance visualization).

I used Flask as the web framework to build a RESTful backend, SQLite for database management via Flask-SQLAlchemy, and Plotly for interactive charts.

The development process involved:

- Designing the database schema (ER diagram) to store users, subjects, chapters, quizzes, questions, and scores.
- Implementing user authentication for admins (role 0) and users (role 1) with login/signup functionality.
- Creating routes and templates for admin dashboards (subject/chapter/quiz/question management), user dashboards (quiz attempts, score tracking), and summary pages with Plotly charts.
- Ensuring responsiveness and styling with Bootstrap and custom CSS for an attractive UI.

Challenges included handling session management, ensuring chart generation, and maintaining database consistency, which were resolved through debugging, testing, and iterative development.

3. Frameworks and Libraries Used

- **Flask:** Web framework for building the backend and handling routes, templates, and sessions.
- **Flask-SQLAlchemy:** ORM for interacting with the SQLite database, managing models (User_Info, Subject, Chapter, Quiz, Question, Score).
- **Plotly:** Python library for generating interactive bar and pie charts to visualize quiz attempts and scores.
- **SQLite:** Lightweight relational database for storing project data.
- **Bootstrap 5:** CSS framework (via CDN) for responsive, attractive UI design (e.g., forms, cards, navbar).
- **Jinja2:** Templating engine for rendering HTML templates dynamically.

Description: The database schema includes the following tables and their relationships:

1. **User_Info:**

- a. Fields: id (PK), email, password, role, full_name, qualifications, dob
- b.

Relationships: One-to-Many with Score (a user can have multiple scores).

2. **Subject:**

- a. Fields: id (PK), name, description
- b.

Relationships: One-to-Many with Chapter (a subject can have multiple chapters).

3. **Chapter:**

- a. Fields: id (PK), name, description, subject_id (FK to Subject)
- b.

Relationships: One-to-Many with Quiz (a chapter can have multiple quizzes).

4. **Quiz:**

- a. Fields: id (PK), date_of_quiz, time_duration, remarks, chapter_id (FK to Chapter)
- b. Relationships: One-to-Many with Question (a quiz can have multiple questions), One-to-Many with Score (a quiz can have multiple scores from users).

5. **Question:**

- a. Fields: id (PK), question_statement, option1, option2, option3, option4, correct_option, quiz_id (FK to Quiz)

6. **Score:**

- a. Fields: id (PK), total_scored, time_stamp_of_attempt, user_id (FK to User_Info), quiz_id (FK to Quiz)

5. API Resource Endpoints (If Any)

My project is primarily a web application with Flask routes rather than a REST API, list the key Flask routes as “endpoints”:

- **GET /:** Renders the index.html landing page.
- **GET/POST /login:** Handles user login, redirects to admin_dashboard or user_dashboard based on role.
- **GET/POST /signup:** Handles user registration, redirects to login.
- **GET /admin/<name>:** Displays the admin dashboard with subjects, chapters, quizzes, and charts.
- **GET /user/<name>:** Displays the user dashboard with subjects, quizzes, and charts.
- **GET /summary/<name>:** Shows user summary with charts.
- **GET /view_quiz/<int:quiz_id>/<name>:** Displays quiz details.
- **GET/POST /quiz_attempt/<quiz_id>/<name>:** Allows users to attempt a quiz, submits scores.
- **GET /quiz_scores/<name>:** Shows user quiz scores.
- **GET/POST /search/<name>:** Searches subjects, chapters, quizzes, and users for admins.
- **GET/POST /subject/<name>:** Adds a new subject for admins.
- **GET/POST /edit_subject/<id>/<name>:** Edits a subject for admins.
- **POST /delete_subject/<id>/<name>:** Deletes a subject for admins.
- **GET/POST /chapter/<subject_id>/<name>:** Adds a new chapter for admins.
- **GET/POST /edit_chapter/<id>/<name>:** Edits a chapter for admins.
- **POST /delete_chapter/<id>/<name>:** Deletes a chapter for admins.
- **GET /quiz/<name>:** Displays the quiz dashboard for admins.
- **GET/POST /add_quiz/<name>:** Adds a new quiz for admins.
- **GET/POST /edit_quiz/<quiz_id>/<name>:** Edits a quiz for admins.
- **POST /delete_quiz/<int:quiz_id>/<name>:** Deletes a quiz for admins.

- **GET/POST /quiz/<int:quiz_id>/questions:** Adds a new question to a quiz for admins.
- **GET/POST /edit_question/<int:id>:** Edits a question for admins.
- **POST /delete_question/<int:id>:** Deletes a question for admins.

6. Drive Link of the Presentation Video

- **Video Link:**
<https://drive.google.com/drive/folders/1nxmtKXh3DFaBGmbpTbo9LfKymzrL1fVQ?usp=sharing>